Experiment12

1) Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <time.h>

int total_items;                              /* items to transfer */
int *buffer, buffer_size;
int in = 0, out = 0, count = 0, violations = 0;
int num_prod, num_cons, verbose;
sem_t empty_slots, full_slots;               /* counting semaphores */
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

typedef struct { int id; long sum; } worker_t;

void *producer(void *arg)
{
    worker_t *w = arg;
    int per = total_items / num_prod;
    for (int k = 1; k <= per; k++) {
        int item = w->id * per + k;          /* unique item number */
        sem_wait(&empty_slots);              /* wait for a free slot */
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        in = (in + 1) % buffer_size;
        count++;
        if (count > buffer_size) violations++;
        if (verbose)
            printf("Producer %d produced %3d  (items in buffer: %d)\n",
                   w->id, item, count);
        pthread_mutex_unlock(&mutex);
        sem_post(&full_slots);               /* signal item available */
    }
    return NULL;
}

void *consumer(void *arg)
{
    worker_t *w = arg;
    int per = total_items / num_cons;
    for (int k = 0; k < per; k++) {
        sem_wait(&full_slots);               /* wait for an item */
        pthread_mutex_lock(&mutex);
        int item = buffer[out];
        out = (out + 1) % buffer_size;
        count--;
        if (count < 0) violations++;
        if (verbose)
            printf("Consumer %d consumed %3d  (items in buffer: %d)\n",
                   w->id, item, count);
        pthread_mutex_unlock(&mutex);
        sem_post(&empty_slots);              /* signal free slot */
        w->sum += item;
    }
    return NULL;
}

/* Runs one experiment and returns throughput (items/second) */
double run(int bsize, int np, int nc, int items, int show)
{
```

```
    total_items = items;
    buffer_size = bsize; num_prod = np; num_cons = nc; verbose = show;
    in = out = count = violations = 0;
    buffer = malloc(sizeof(int) * bsize);
    sem_init(&empty_slots, 0, bsize);        /* all slots empty at start */
    sem_init(&full_slots, 0, 0);             /* no items at start        */

    pthread_t p[16], c[16];
    worker_t pw[16], cw[16];
    struct timespec t1, t2;
    clock_gettime(CLOCK_MONOTONIC, &t1);
    for (int i = 0; i < np; i++) {
        pw[i].id = i; pw[i].sum = 0;
        pthread_create(&p[i], NULL, producer, &pw[i]);
    }
    for (int i = 0; i < nc; i++) {
        cw[i].id = i; cw[i].sum = 0;
        pthread_create(&c[i], NULL, consumer, &cw[i]);
    }
    for (int i = 0; i < np; i++) pthread_join(p[i], NULL);
    for (int i = 0; i < nc; i++) pthread_join(c[i], NULL);
    clock_gettime(CLOCK_MONOTONIC, &t2);

    double el = (t2.tv_sec - t1.tv_sec) + (t2.tv_nsec - t1.tv_nsec) / 1e9;
    long consumed = 0;
    for (int i = 0; i < nc; i++) consumed += cw[i].sum;
    long expected = (long)total_items * (total_items + 1) / 2;

    if (!show) {
        printf("%-8d %-12.4f %-14.0f %-12s %-10d\n", bsize, el, total_items / el,
               consumed == expected ? "CORRECT" : "WRONG", violations);
    }
    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);
    free(buffer);
    return total_items / el;
}

int main(void)
{
    int sizes[] = {1, 5, 10, 50, 100};

    printf("DEMO: 1 producer, 1 consumer, buffer size 3, 6 items\n");
    printf("-------------------------------------------------------------\n");
    run(3, 1, 1, 6, 1);

    printf("\nBENCHMARK: 2 producers, 2 consumers, 200000 items\n");
    printf("%-8s %-12s %-14s %-12s %-10s\n",
           "Buffer", "Time(s)", "Items/sec", "Checksum", "Violations");
    printf("-------------------------------------------------------------\n");
    for (int i = 0; i < 5; i++)
        run(sizes[i], 2, 2, 200000, 0);
    return 0;
}
```

OUTPUT :

```
charith@DESKTOP-RBQQQSS:~$ nano exp12.1.c
charith@DESKTOP-RBQQQSS:~$ gcc exp12.1.c -o exp12.1 -lpthread
charith@DESKTOP-RBQQQSS:~$ ./exp12.1
DEMO: 1 producer, 1 consumer, buffer size 3, 6 items
------------------------------------------------------------
Producer 0 produced   1  (items in buffer: 1)
Producer 0 produced   2  (items in buffer: 2)
Producer 0 produced   3  (items in buffer: 3)
Consumer 0 consumed   1  (items in buffer: 2)
Producer 0 produced   4  (items in buffer: 3)
Consumer 0 consumed   2  (items in buffer: 2)
Producer 0 produced   5  (items in buffer: 3)
Consumer 0 consumed   3  (items in buffer: 2)
Producer 0 produced   6  (items in buffer: 3)
Consumer 0 consumed   4  (items in buffer: 2)
Consumer 0 consumed   5  (items in buffer: 1)
Consumer 0 consumed   6  (items in buffer: 0)

BENCHMARK: 2 producers, 2 consumers, 200000 items
Buffer   Time(s)      Items/sec      Checksum     Violations
------------------------------------------------------------
1        0.5581       358375         CORRECT      0
5        0.1666       1200769        CORRECT      0
10       0.1190       1680864        CORRECT      0
50       0.0780       2563187        CORRECT      0
100      0.0694       2880359        CORRECT      0
charith@DESKTOP-RBQQQSS:~$
```

2) Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

Code :

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
#include <time.h>

#define N 3                                     /* threads and resources */
#define TIMEOUT 3          /* seconds to wait before reporting deadlock */

pthread_mutex_t R[N];                           /* shared resources R0, R1, R2 */
pthread_barrier_t barrier;
int ordered = 0;                                /* 0 = deadlock mode, 1 = resource ordering */

/* lock with a timeout, only so that the demo can report the deadlock and exit */
int lock_timeout(pthread_mutex_t *m)
{
    struct timespec ts;
    clock_gettime(CLOCK_REALTIME, &ts);
    ts.tv_sec += TIMEOUT;
    return pthread_mutex_timedlock(m, &ts);
}

void *worker(void *arg)
{
```

```
    int id = (int)(long)arg;
    int a = id, b = (id + 1) % N;             /* needs R[id] and R[id+1] */

    if (ordered && a > b) {     /* PREVENTION: always lock lower number first */
        int t = a; a = b; b = t;
    }

    pthread_mutex_lock(&R[a]);
    printf("Thread %d acquired R%d\n", id, a);
    sleep(1);                   /* let other threads grab their first resource */

    printf("Thread %d waiting for R%d (holding R%d)\n", id, b, a);
    if (lock_timeout(&R[b]) != 0) {
        printf("Thread %d: DEADLOCK - blocked for %d s waiting for R%d while holding R%d\n",
               id, TIMEOUT, b, a);
        pthread_barrier_wait(&barrier);       /* everyone reports before anyone releases */
        pthread_mutex_unlock(&R[a]);
        return NULL;
    }

    printf("Thread %d acquired R%d -> working with R%d and R%d\n", id, b, a, b);
    usleep(200000);
    pthread_mutex_unlock(&R[b]);
    pthread_mutex_unlock(&R[a]);
    printf("Thread %d released R%d and R%d (finished)\n", id, a, b);
    return NULL;
}

int main(int argc, char *argv[])
{
    if (argc != 2 || (strcmp(argv[1], "deadlock") && strcmp(argv[1], "ordered"))) {
        fprintf(stderr, "Usage: %s deadlock | ordered\n", argv[0]);
        return 1;
    }
    ordered = strcmp(argv[1], "ordered") == 0;

    printf("Mode: %s\n", ordered ? "DEADLOCK PREVENTION (resource ordering)"
                                 : "DEADLOCK DEMONSTRATION (circular wait)");
    printf("T0 needs R0,R1 | T1 needs R1,R2 | T2 needs R2,R0\n");
    printf("------------------------------------------------\n");

    for (int i = 0; i < N; i++) pthread_mutex_init(&R[i], NULL);
    pthread_barrier_init(&barrier, NULL, N);

    pthread_t t[N];
    for (long i = 0; i < N; i++) pthread_create(&t[i], NULL, worker, (void *)i);
    for (int i = 0; i < N; i++) pthread_join(t[i], NULL);

    printf("------------------------------------------------\n");
    if (ordered)
        printf("All threads completed - no deadlock (circular wait is impossible).\n");
    else
        printf("Program ended only because of the timeout - "
               "without it all threads wait forever.\n");
    return 0;
}
```

OUTPUT :

```
charith@DESKTOP-RBQQQSS:~$ nano exp12.2.c
charith@DESKTOP-RBQQQSS:~$ gcc exp12.2.c -o exp12.2 -lpthread
charith@DESKTOP-RBQQQSS:~$ ./exp12.2 deadlock
Mode: DEADLOCK DEMONSTRATION (circular wait)
T0 needs R0,R1 | T1 needs R1,R2 | T2 needs R2,R0
------------------------------------------------
Thread 0 acquired R0
Thread 1 acquired R1
Thread 2 acquired R2
Thread 1 waiting for R2 (holding R1)
Thread 2 waiting for R0 (holding R2)
Thread 0 waiting for R1 (holding R0)
Thread 0: DEADLOCK - blocked for 3 s waiting for R1 while holding R0
Thread 2: DEADLOCK - blocked for 3 s waiting for R0 while holding R2
Thread 1: DEADLOCK - blocked for 3 s waiting for R2 while holding R1
------------------------------------------------
Program ended only because of the timeout - without it all threads wait forever.
charith@DESKTOP-RBQQQSS:~$ ./exp12.2 ordered
Mode: DEADLOCK PREVENTION (resource ordering)
T0 needs R0,R1 | T1 needs R1,R2 | T2 needs R2,R0
------------------------------------------------
Thread 0 acquired R0
Thread 1 acquired R1
Thread 1 waiting for R2 (holding R1)
Thread 1 acquired R2 -> working with R1 and R2
Thread 0 waiting for R1 (holding R0)
Thread 0 acquired R1 -> working with R0 and R1
Thread 1 released R1 and R2 (finished)
Thread 0 released R0 and R1 (finished)
Thread 2 acquired R0
Thread 2 waiting for R2 (holding R0)
Thread 2 acquired R2 -> working with R0 and R2
Thread 2 released R0 and R2 (finished)
------------------------------------------------
All threads completed - no deadlock (circular wait is impossible).
charith@DESKTOP-RBQQQSS:~$
```